

DS/CMPSC 442: Artificial Intelligence
Fall 2026
Project-1 Solve the River Crossing Puzzle
(Due: Sep 25th 11:59 PM)
Total Marks: 40
Extra Credit Marks: 10



Figure.1 A simple example of the river crossing puzzle

Project Description

The **River Crossing Puzzle** is a classic AI problem. In this puzzle, you must transport a group of characters across a river using a boat, subject to rules and constraints.

Scenario (Missionaries & Cannibals variant):

- There are **3 missionaries** and **3 cannibals** on the left bank of the river.
- The boat can carry **at most 2 people** at a time, and has to carry **at least one person** at a time.
- At no point on either bank should cannibals outnumber missionaries (if missionaries are present).

The **goal state** is to move all missionaries and cannibals to the right bank without breaking the rules.

You will implement **search algorithms (DFS, BFS, UCS, A*)** in **Python 3.9** to solve this problem under different conditions.

Files to Submit: Submit a **.zip file** containing exactly four files: (See Table.1 for file name and explanation).

File Name	Explanation
File-1: solution_q1.py	Implementations for all Q1 coding tasks. Must run with <code>python solution_q1.py</code> .
File-2: solution_q2.py	Implementations for all Q2 coding tasks. Must run with <code>python solution_q2.py</code> .
File-3: solution_q3.py	Implementations for all Q3 coding tasks. Must run with <code>python solution_q3.py</code> .
File-4: proj-1-submission.pdf	Written answers to writing component questions.

Table 1. Submission files for project-1

The program must read from input.txt, which specifies the **initial state** of the puzzle

- The state will be represented as:

M_left,C_left,M_right,C_right,Boat

where `M_left` and `C_left` are the number of missionaries and cannibals on the left bank, `M_right` and `C_right` are the number of missionaries and cannibals on the right bank, and Boat is either `L` (left) or `R` (right).

- Example initial state in which all three missionaries and cannibals are on the left bank, and the boat is standing on the left bank as well:

3, 3, 0, 0, L

As the solution of this project, your code for each question should print three things: (i) the **sequence of actions your algorithm finds**, total cost associated with this sequence of actions (if applicable), and number of node expansions that occur in your tree search.

Evaluation: Your code will be graded manually by TA for technical correctness. To foster the grading process, they will directly check the output printed in the terminal after running the command “python solution_q1.py”. So make sure your code could be running in the terminal smoothly before making the submission.

Academic Dishonesty: We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else’s code and submit it with minor changes, we will know. At the same time, do not attempt to use tools like ChatGPT, LLaMa, Gemini, or any other large language model. Our cheating detectors are quite hard to fool, so please don’t try. We trust you all to submit your own work only; please don’t let us down. If you do, we will pursue the strongest consequences available to us.

Question-1 DFS and BFS for the River Crossing Puzzle

Q1.1.1.a DFS (4 marks)

- Implement DFS algorithm that works with starting state that is read in from an input file called `input.txt`. The format of the state should be the same as what we mention above.
- Output: path of states returned, total cost (where cost is defined in terms of number of actions), and number of node expansions.

Q1.1.b BFS (4 marks)

- Implement a BFS that works with starting state that is read in from an input file called `input.txt`. The format of the state should be the same as what we mention above.
- Output: path of states returned, total cost (where cost is defined in terms of number of actions), and number of node expansions.

Expected terminal output format:

```
The solution of Q1.1.a (DFS) is:
Solution Path: <Path>
Total cost = <number>
Number of node expansions = <number>

The solution of Q1.1.b (BFS) is:
Solution Path: <Path>
Total cost = <number>
Number of node expansions = <number>
```

Q1.2 The Writing Component (2 marks)

Using the initial state `3, 3, 0, 0, L`:

1. Report the **solution path** and **number of node expansions** for DFS and BFS.
 2. Write a short **free-form explanation (1–2 paragraphs)** comparing DFS and BFS in the context of this problem:
 - What trade-offs do you observe?
 - Why does BFS guarantee an optimal solution (if one exists), while DFS may fail to do so?
 - What do the node expansion counts tell you about efficiency vs. completeness?
-

Questions-2 Uniform Cost Search (UCS) with Non-Uniform Action Costs

We now extend the river crossing puzzle to include **non-uniform action costs**, so that path *cost* may differ from path *length*. UCS should always return a **least-cost** solution.

State format remains:

M_left, C_left, M_right, C_right, Boat
--

Q2.1 The Coding Component (UCS) (8 marks)

- Implement **Uniform Cost Search** in `solution_q2.py`.
- Output: path of states returned, total cost, and number of node expansions.
- Your program must support **two different cost models**:

A. Cost Model A — Cost by Passenger Type

Story Motivation: Missionaries carry heavy books and supplies, while cannibals travel light. Every missionary on the boat adds more rowing effort. Hence, it is twice as costly to carry a missionary as compared to a cannibal.

Cost = **2 units per missionary** + **1 unit per cannibal**.

Examples:

- Move {M, M} → cost 4
- Move {C, C} → cost 2
- Move {M, C} → cost 3

B. Cost Model B — Cost by Boat Direction

Story Motivation: The river current flows right → left. Going against the current (left → right) is harder, while coming back (right → left) is easier.

- Left → Right trip = cost 2
- Right → Left trip = cost 1

Running & Output Format

Your script should allow selecting the cost model (e.g., via command-line flag or extra line in `input.txt`).

Expected output format:

The solution of Q2.1 (UCS, cost model A) is: Solution Path: Total cost = <number> Number of node expansions = <number> The solution of Q2.1 (UCS, cost model B) is: Solution Path: Total cost = <number> Number of node expansions = <number>
--

Q2.2 The Writing Component (2 marks)

Using initial state 3, 3, 0, 0, L:

1. Run UCS with **Cost Model A** and **Cost Model B**, report:
 - o Solution path
 - o Total cost
 - o Number of node expansions
 2. Compare with BFS:
 - o Does UCS choose a different sequence of actions?
 - o How does **total cost** (UCS) relate to **path length** (BFS)?
 - o Which cost model leads to more/fewer node expansions?
 3. In **1–2 paragraphs**, explain why UCS is necessary with non-uniform costs, and discuss the trade-off between **least cost** and **fewest steps**.
-

Question 3: A* Search with Admissible Heuristics

In this part, you will extend UCS to A* search using **Cost Model A**.

Q3.1 The Coding Component (A* with Heuristics)

We will use the following heuristics with A*

Heuristic 1 — Passenger Weight Remaining

$$h_1(s) = 2M_{\text{left}} + 1C_{\text{left}}$$

Heuristic 2 — Trip-Packing Lower Bound

$$h_2(s) = \lceil \frac{2M_{\text{left}} + 1C_{\text{left}}}{3} \rceil$$

Expected terminal output format:

```
The solution of Q3.1 (Heuristic 1) is:
Solution Path: <Path>
Total cost = <number>
Number of node expansions = <number>

The solution of Q3.1 (Heuristic 2) is:
Solution Path: <Path>
Total cost = <number>
Number of node expansions = <number>
```

Q3.2 The Writing Component

Part A — Admissibility Proofs (6 marks)

For each heuristic (h_1 , h_2):

- Prove or disprove that it is **admissible**.
- If admissible, explain why it never overestimates.
- If not, provide a counterexample.

Part B — Implementation and Comparison (8 marks)

1. Implement A* with h_1 and h_2 .
2. Run both on the initial state 3, 3, 0, 0, L
3. Record results in a table:

Heuristic	Solution Path (brief)	Total Cost	Node Expansions	Same Cost as UCS?
h_1				Yes/No
h_2				Yes/No

4. Compare node expansions: which heuristic was more efficient, and why?

Part C — Designing a Tighter Heuristic (6 marks)

Propose your own heuristic h_3 that is **admissible** and which dominates h_1 or h_2 . Note, we are not looking for the $\max(h_1, h_2)$ heuristic. We want you to get creative with your heuristic.

- Define it clearly.
- Argue why it is admissible.
- Implement h_3 in A* and compare node expansions against h_1 and h_2 by adding a third row in the table above (this completed table should be added to proj-1-submission.pdf (the written file that you will submit with your project).

Q3.3 Extra Credit — Consistency Proofs (10 marks)

For each heuristic (h_1 , h_2 , and your h_3):

- Prove or disprove that it is **consistent**.
- Recall: h is consistent if for every edge $s \rightarrow s'$ with cost c :

$$h(s) \leq c + h(s')$$